# Technical Handover: StomaML

Mia Lindblad, Valtteri Ahola, Markus Tarma, Soumyadipta Kar, Miika Mäki

March 2026

## Contents

**1 Overview** **3**

**2 Running the application** **3**
2.1 Recommended method: Docker Compose . . . 3
2.1.1 In case of failure . . . 4
2.2 Running without Docker . . . 5
2.2.1 Backend . . . 5
2.2.2 Frontend . . . 6

**3 Output fields and calculations** **6**
3.1 Image-level summary output . . . 6
3.2 Stomata-level metadata output . . . 7
3.3 Processing order . . . 8

**4 System structure** **8**

**5 Repository structure** **8**

**6 Frontend** **8**
6.1 Role of the frontend . . . 8
6.2 Frontend files . . . 8
6.3 Frontend behaviour . . . 9
6.4 User inputs in the frontend . . . 9
6.5 Implementation details of `main.py` . . . 9
6.6 Layout and interaction design . . . 10
6.7 Batch processing behaviour . . . 11
6.8 Displayed results . . . 11
6.9 Progress bar and remaining time estimation . . . 11
6.10 Configuration handling . . . 11
6.11 Containerisation and startup . . . 12
6.12 Streamlit configuration . . . 12
6.13 Dependencies . . . 12
6.14 Maintenance considerations . . . 13

**7 Backend** **13**
7.1 Role of the backend . . . 13
7.2 Backend files . . . 13
7.3 FastAPI application structure . . . 14
7.4 Trusted host middleware . . . 14
7.5 API endpoints . . . 14
7.6 Returned API response structure . . . 15
7.7 Model manager . . . 15

7.8 Inference pipeline . . . 16
7.9 Mask handling and post-processing . . . 16
7.10 Image-level calculations . . . 17
7.11 Stomata-level calculations . . . 17
7.12 Functions in `utils/inference.py` . . . 17
7.13 Error handling . . . 17
7.14 Runtime defaults and constants . . . 18
7.15 Dependencies . . . 18
7.16 Containerisation . . . 18
7.17 Maintenance considerations . . . 19

**8 Deployment** **19**
8.1 OpenShift . . . 19
8.2 Local use and testing . . . 20

**9 Checks after changes** **20**

# 1 Overview

This document covers how StomaML works, how to run it, and how it is put together under the hood. The scientific background and project motivation are covered in the final report, so they are not repeated here.

The handover is written with two audiences in mind. The first is end users or future project members who mainly need to run the application and understand what goes in and what comes out. The second is developers or maintainers who may need to debug things, redeploy, or build on top of the existing system. Because of this, the earlier sections focus on practical usage, while the later sections go into more technical detail about the software itself.

The source code is available at https://github.com/valttteri/StomaML.

# 2 Running the application

## 2.1 Recommended method: Docker Compose

The simplest way to run StomaML locally is with Docker Compose. It starts both the frontend and backend at the same time and removes the need to set them up individually.

The file `docker-compose.yaml` in the repository defines the local multi-container setup. It currently includes two services:

- `backend` – built from `./backend` and exposed on port `8080`
- `frontend` – built from `./frontend` and exposed on port `8050`

The frontend service also receives the environment variable

```
API_URL=http://backend:8080/predict
```

which lets the frontend container reach the backend through Docker Compose's internal networking. In other words, the frontend does not talk to `localhost` when running inside Compose – it uses the service name `backend` instead.

To use this method, you need:

- the full project repository,
- Docker and Docker Compose installed,
- the trained model weights placed at `backend/models/weights.pt`.

Open a terminal in the directory containing `docker-compose.yaml` and run:

```
docker compose up --build
```

Docker will then build separate images for the frontend and backend using their respective `Dockerfile`s. Once everything is up, the frontend should be reachable at:

```
http://127.0.0.1:8050
```

Docker might also show `0.0.0.0:8050` in its output, but for opening the app in a browser, `http://127.0.0.1:8050` or `http://localhost:8050` is usually the safest bet.

Once the interface loads in the browser, the application is ready. When done, press `Ctrl+C` in the same terminal to stop the containers.

Figure 1 shows what a typical successful startup looks like.

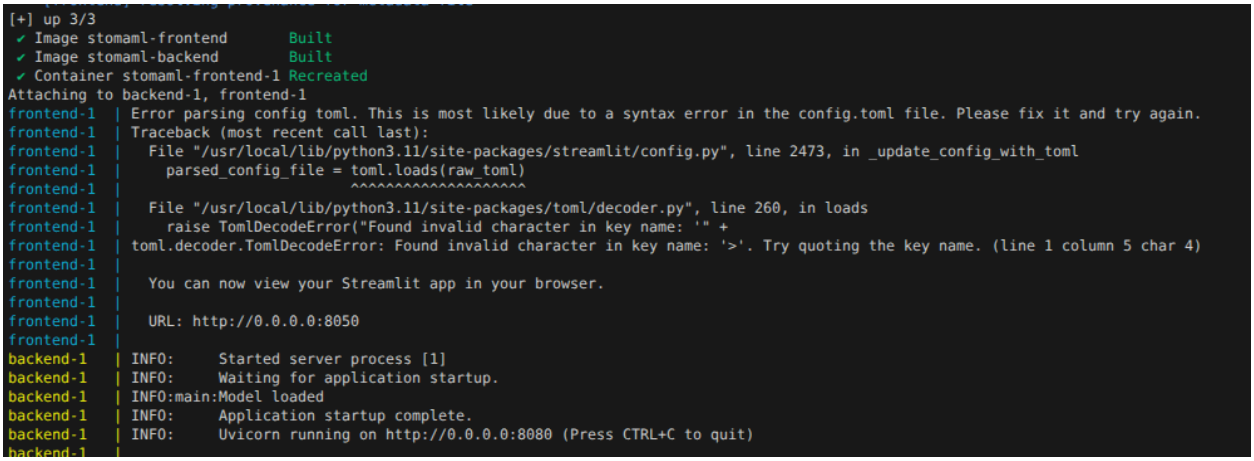


Figure 1: Example of a successful Docker Compose startup. Some warning messages may appear but they do not prevent local use. The frontend URL is shown in the terminal output.

### 2.1.1 In case of failure

Some files are sensitive to line endings. A `.gitattributes` file is included to keep the repository on LF endings, but in practice shell scripts can still end up with CRLF on some systems or editors – especially right after cloning.

If you see something like:

```
frontend-1 | exec /entrypoint.sh:  no such file or directory
```

the most likely cause is that `frontend/entrypoint.sh` has been saved with CRLF instead of LF.

In Visual Studio Code, you can fix this by opening the file, finding the `CRLF` indicator in the lower right corner, switching it to `LF`, and saving. Sometimes the editor needs a restart for the change to fully take effect.

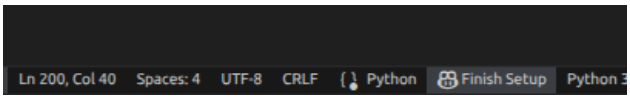


Figure 2: The CRLF indicator in the lower right corner of Visual Studio Code.

If a container does not start, checking its logs is a good first step:

```
docker compose logs frontend
```

or

```
docker compose logs backend
```

You can also run:

```
docker compose ps
```

This shows whether containers are running, which ports are mapped, and whether something exited right after starting.

Another common issue is port conflicts. If something else is already using one of the mapped ports, startup will fail. The ports used in this project are:

- `8080` for the backend
- `8050` for the frontend

Either stop whatever is using the port, or change the port mapping in `docker-compose.yaml`.

## 2.2 Running without Docker

If Docker Compose is not an option, the frontend and backend can be run separately in local Python environments. This is more suited for development or troubleshooting than everyday use.

It is a good idea to use separate virtual environments for the frontend and backend so that their dependencies stay isolated. You can create one with:

```
python -m venv .venv
```

After creating it, activate the environment before installing anything. The activation command depends on your OS and terminal.

### 2.2.1 Backend

To start the backend, open a terminal, go to the backend directory, remember to activate the virtual environment, install the dependencies, and launch the FastAPI app:

```
cd backend
```

```
pip install -r requirements.txt
```

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Make sure the model weights file exists at:

```
backend/models/weights.pt
```

If the backend starts correctly, you can check its health endpoint at:

```
http://127.0.0.1:8000/health
```

Note that the Dockerised backend uses port `8080`, while the example above uses `8000`. This is fine, but keep it in mind when debugging connectivity issues.

#### 2.2.2 Frontend

Start the frontend in a second terminal (remember to activate the frontend virtual environment):

```
cd frontend

pip install -r requirements.txt

streamlit run main.py
```

Before launching, make sure the frontend's configuration points to the right backend endpoint. For local development, that usually means setting `API_URL` to:

```
http://127.0.0.1:8000/predict
```

The `/predict` part matters – the frontend sends requests directly to the prediction endpoint, not just the backend root URL. The confidence threshold and image scale are also passed along as query parameters in the request URL, for example:

```
http://127.0.0.1:8000/predict?conf=0.5&scale=0.2174
```

These are added automatically by the frontend when it sends a request, so there is no need to set them manually, but it is good to know they are part of the call when debugging.

When running things this way, the backend has to be up before the frontend can talk to it. If either service tries to use a port that is already taken, it will not start until the conflict is resolved.

# 3 Output fields and calculations

After processing, the application returns results for each uploaded image. These include summary values like stomata count and density measurements, as well as more detailed per-detection information. This section is particularly useful for end users, since it explains what each value in the CSV and JSON metadata files means and how it is computed. It is also relevant for future developers, because the calculation logic needs to stay consistent over time.

The backend returns one result object per uploaded image. Each result contains image-level information and a list of per-stomata metadata entries.

## 3.1 Image-level summary output

The `density_info` object contains the following fields:

- `total_pixels` – the total number of pixels in the image, calculated as:

  $$\texttt{image height} \times \texttt{image width}$$

- `non_countable_pixels` – the number of pixels belonging to the union of all masks classified as non-countable area. In the current implementation, any detection whose class name is listed in `NON_COUNTABLE_CLASSES` contributes to this mask.
- `countable_pixels` – the number of pixels not covered by the non-countable mask. Simply the total pixels minus the non-countable ones.
- `stomata_count` – the number of detected stomata that made it into the final metadata list. In practice, this is just the length of the `stomata` list.
- `stomatal_density_per_px2` – stomatal density in stomata per pixel squared:

  $$\frac{\texttt{stomata\_count}}{\texttt{countable\_pixels}}$$

  If `countable_pixels` is zero, this field is set to zero to avoid division by zero.

- `um_per_px` – the image scale in micrometers per pixel, as provided by the user.
- `stomatal_density_per_mm2` – stomatal density converted to stomata per square millimetre. The conversion works by first turning the countable pixel area into $\text{mm}^2$ using the image scale, and then dividing the stomata count by that area:

$$\text{countable area in mm}^2 = \texttt{countable\_pixels} \times \left(\frac{\texttt{um\_per\_px}}{1000}\right)^2$$

$$\texttt{stomatal\_density\_per\_mm2} = \frac{\texttt{stomata\_count}}{\text{countable area in mm}^2}$$

- `avg_stomatal_length_px`, `avg_stomatal_length_um` and `avg_stomatal_length_mm` – Average length of the stomata in pixels, micrometers and millimeters, respectively. The length is estimated by fitting an ellipse on the mask using OpenCV's cv2.fitEllipse. If the mask has less than five points, fitting an ellipse is not possible and these values are `None`.
- `avg_stomatal_size_px` and `avg_stomatal_size_um2` – Average area of the stomata in pixels and square micrometers.
- `avg_closest_stomata_distance_px`, `avg_closest_stomata_distance_um` and `avg_closest_stomata_distance_mm` – The average Euclidean distance from a stomata to the nearest stomata in pixels, micrometers and millimiters, respectively. The Euclidean distance is measured between the centroids of the stomata. If there is only one stomata in the image, these are `None`.

## 3.2 Stomata-level metadata output

Each detected stomata that passes filtering gets its own metadata dictionary. The fields are:

- `id` – a running index for the stomata within the image.
- `confidence` – the model's predicted confidence for this detection.
- `pixel_area` – the number of pixels in the stomata's binary mask.
- `area_um2` – the mask area converted to square micrometers:

$$\texttt{pixel\_area} \times \texttt{um\_per\_px}^2$$

- `bbox_xyxy` – the tight bounding box around the mask, given as $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$.
- `width_px` and `height_px` – bounding box width and height in pixels.
- `centroid_x` and `centroid_y` – the mean of all mask pixel coordinates.
- `length_major_axis_px` and `length_minor_axis_px` – estimated from an ellipse fitted to the mask using OpenCV's `cv2.fitEllipse`. If the mask has fewer than five points, ellipse fitting is not possible and these values are `None`.
- `length_major_axis_um` – the major axis length converted to micrometers.
- `fraction_in_countable_area` – the fraction of the stomata mask that falls within countable area:

$$\frac{\text{stomata mask pixels inside countable area}}{\texttt{pixel\_area}}$$

  A stomata is only kept if this value is at least 0.5.
- `closest_stomata_distance_px`, `closest_stomata_distance_um`, and `closest_stomata_distance_mm` – the Euclidean distance from this stomata's centroid to the nearest other stomata's centroid, in pixels, micrometers, and millimetres respectively. If there is only one stomata in the image, these are `None`.

### 3.3 Processing order

To understand the output, it helps to know the order of operations:

- Detected stomata masks are first collected as candidates.
- Each candidate is checked against the countable region. Only those with at least 50% of their area inside countable area become final metadata entries.
- Nearest-neighbour distances are calculated after all entries have been finalised, using centroid-to-centroid Euclidean distance.
- If the model returns no masks at all, the backend returns an empty stomata list and an empty `density_info` dictionary.

## 4 System structure

StomaML has two main parts: a Streamlit frontend and a FastAPI backend. The frontend provides the user interface, handles file uploads, and shows results. The backend holds the machine learning model and all the inference logic.

In practice, the user only interacts with the frontend. When an analysis is started, the frontend sends the uploaded files and chosen parameters to the backend API. The backend loads the trained model, runs segmentation, turns the detections into structured output, and sends everything back for the frontend to display and offer for download.

This split keeps the interface simple for end users while letting the ML and API logic live in their own maintainable part of the codebase.

## 5 Repository structure

The repository mirrors this division. `frontend/` contains the Streamlit application, and `backend/` contains the FastAPI service and inference logic. Trained model weights go in `backend/models/weights.pt`. The `docker-compose.yaml` at the root defines the local multi-container setup, and each service has its own `Dockerfile` for containerised deployment.

This layout makes it easy to work on the UI and the model-serving logic independently.

## 6 Frontend

### 6.1 Role of the frontend

The frontend is the part of the application that users actually see. It is built with Streamlit so that everything can be done in a browser rather than on the command line. Its job is intentionally narrow: collect user input, send requests to the backend, and present the results. It does not do any machine learning computation.

This separation matters for maintenance. The frontend handles interaction and presentation only. All model loading, inference, post-processing, and scientific calculations live in the backend. So if you need to change the analysis logic, do it in the backend. If you need to change how things look or what the user sees, do it in the frontend.

### 6.2 Frontend files

The frontend code lives in `frontend/`. The main files are:

- `main.py` – the main Streamlit application. It defines the UI, handles file upload, sends inference requests to the backend, tracks progress, and renders the results and download buttons.

- `config.py` – loads the backend API URL from the `API_URL` environment variable, keeping deployment-specific addresses out of the main application code.
- `utils.py` – small helper classes: a wrapper around Streamlit's progress bar and a utility for estimating remaining processing time during batch inference.
- `requirements.txt` – Python dependencies for the frontend.
- `Dockerfile` – defines the container image build for the frontend.
- `entrypoint.sh` – the container startup script. It sets up a Streamlit configuration directory and then launches the application.

There is also a Streamlit configuration file at `.streamlit/config.toml`, discussed further in the Streamlit configuration subsection below.

## 6.3 Frontend behaviour

The frontend is a straightforward single-page application. The workflow goes like this:

1. The user uploads one or more image files.
2. The user sets two numerical values: image scale and confidence threshold.
3. The user presses the analysis button.
4. The frontend sends the images and parameters to the backend.
5. The frontend receives the JSON response, pulls out the image-level and stomata-level results, and shows the summary in a table.
6. The user can download the summary as a CSV and the per-stomata metadata as a JSON file.

Accepted file types are `jpg`, `jpeg`, and `png`. Multiple files can be uploaded and processed in one go. The details of how batch processing works are described in the batch processing subsection below.

## 6.4 User inputs in the frontend

Besides the uploaded files, the interface exposes two numeric parameters.

**Image scale** The image scale is entered as micrometers per pixel. The default value is:

$$\frac{10}{46} \approx 0.2174$$

This comes from the project's default calibration assumption that $10\,\mu$m corresponds to 46 pixels. The help text in the UI communicates this to the user. The value is passed unchanged to the backend, which uses it to convert pixel-based measurements into physical units.

**Confidence threshold** The confidence threshold defaults to 0.5, with an accepted range of 0.01 to 0.99 in the frontend. It is sent to the backend as the `conf` query parameter and controls which detections the model pipeline keeps.

## 6.5 Implementation details of `main.py`

The main UI logic lives in `frontend/main.py`. It has three helper functions and one main application function.

`call_api_with_files` Handles communication with the backend. It takes a list of file paths along with the confidence threshold and image scale, packs the files into a multipart request via `requests.post()`, and sends the numeric settings as request parameters. The backend address comes from `API_URL` (imported from `config.py`).

The request timeout is currently set to 120 seconds. If a request fails, the exception is shown to the user through Streamlit and then re-raised. This means errors are not silently swallowed, which is helpful for debugging but may look somewhat technical to non-developer users.

`download_csv` Takes a Pandas DataFrame with the image-level results and creates a Streamlit download button. The file is exported as UTF-8 CSV under the name `stoma_results.csv`. The DataFrame index is included in the output.

`download_metadata` Converts per-stomata metadata into formatted JSON and offers it as a download. The output file is called `metadata.json`. This is meant for anyone who needs more detail than what the summary table shows.

`main` The `main()` function defines the page itself. It sets the title, initialises session state, renders the upload widget and input fields, handles the reset button, runs the analysis when requested, and then shows the results table and download buttons.

Two session state variables are used:

- `inference_done` – a boolean flag for whether processing has finished and results should be shown.
- `file_uploader` – an integer key used to reset the file uploader widget. Incrementing it forces Streamlit to recreate the widget, which clears the selected files.

This reset trick is a common Streamlit pattern and is worth keeping if the upload logic gets changed later.

### 6.6 Layout and interaction design

The frontend uses a compact layout built from Streamlit columns and standard widgets. After uploading files, the page shows:

- a numeric input for image scale,
- a numeric input for confidence threshold,
- a metric showing how many files are uploaded,
- a reset button for clearing the uploads,
- a button for starting analysis.

While the analysis runs, two dynamic elements are displayed:

- a text status showing that analysis is in progress with an estimated time remaining,
- a progress bar that updates after each processed file.

After processing finishes, the page shows a results table under the heading `Analysis results`, followed by download buttons for CSV and JSON output.

The layout is kept simple on purpose. Future groups could certainly improve it, but the current design gets the basic workflow across clearly enough.

## 6.7 Batch processing behaviour

Even though multiple files can be uploaded at once, the frontend does not send them all in a single request. It loops through the files and sends them one at a time. For each image:

1. the uploaded file is written to a temporary file,
2. the temp file path is passed to the API request helper,
3. the backend response is read,
4. image-level and stomata-level data are appended to result arrays,
5. the progress bar and time estimate are updated.

This keeps the frontend logic simple and makes progress tracking straightforward, since each completed image maps directly to one progress update. The downside is that total runtime may be longer than a more advanced batching approach would give, especially with many files.

## 6.8 Displayed results

After all files have been processed, the frontend builds a Pandas DataFrame using the `density_info` fields described in Section 3. A `Filename` column is added and used as the index, so each row corresponds to one image. The table is rendered with `st.dataframe(..., use_container_width=True)` to fill the available page width.

Only image-level summary values are shown directly on the page. The more detailed per-stomata records are only available in the downloaded JSON metadata file.

## 6.9 Progress bar and remaining time estimation

The file `frontend/utils.py` has two small utility classes.

`ProgressBar` A thin wrapper around Streamlit's built-in progress bar. It keeps track of the current value and width, and exposes `render()` for initial display and `update()` for changing the value during processing. It is not a complex component – just a small abstraction to keep the main code tidier.

`RemainingTime` Estimates how much time is left during multi-file inference. It tracks the total file count, the number already processed, and a queue of recent processing times. The logic works as follows:

- for the first few files, it just shows `...` instead of a number,
- it stores up to the 15 most recent inference times,
- it averages those times,
- it multiplies the average by the number of remaining files,
- it formats the result as minutes and seconds.

This is a rough estimate meant purely as a usability convenience. It has no effect on the actual inference.

## 6.10 Configuration handling

As mentioned in the frontend files overview, backend connectivity is configured through `frontend/config.py`, which reads:

```
API_URL = os.getenv("API_URL")
```

The backend address differs between local development, Docker Compose, and OpenShift, so keeping it in an environment variable rather than hardcoded is important. If `API_URL` is missing or wrong, requests will fail as soon as the user tries to run an analysis.

## 6.11 Containerisation and startup

The frontend has its own `Dockerfile`. The container image is currently built from `python:3.11`. The build process:

1. sets `/frontend` as the working directory,
2. copies `requirements.txt` and installs dependencies,
3. copies the frontend source files into the image,
4. copies `entrypoint.sh` to the container root,
5. makes the entrypoint script executable,
6. exposes port 8050,
7. runs the entrypoint script on container start.

The startup script `entrypoint.sh` does a bit of setup before launching Streamlit. It creates a temporary configuration directory at `/tmp/.streamlit`, writes a minimal `config.toml` that turns off usage statistics, sets `HOME` and `STREAMLIT_CONFIG_DIR` accordingly, and then launches:

```
streamlit run main.py --server.address 0.0.0.0 --server.port "${PORT:-8050}"
```

The `${PORT:-8050}` bit means the container defaults to port 8050 but can adapt if a hosting platform injects a different port via the `PORT` variable.

## 6.12 Streamlit configuration

The project includes a Streamlit configuration file at `.streamlit/config.toml`. It has browser and server settings. Telemetry is disabled with:

```
gatherUsageStats = false
```

and CORS is enabled with an explicit list of allowed origins, covering both local development and deployed URLs. This matters because the frontend may not work properly if requests come from a host that Streamlit does not recognise.

One thing to be aware of is that there are currently two related configuration mechanisms:

- the project-level `.streamlit/config.toml`, and
- the runtime configuration generated inside the container by `entrypoint.sh`.

Right now, the entrypoint-generated config only writes the telemetry setting and does not include the CORS origin list from the repository-level file. So Streamlit behaviour might differ slightly depending on whether you run the app from source or through the container.

## 6.13 Dependencies

The frontend dependencies in `frontend/requirements.txt` are:

- `streamlit==1.53.1`
- `python-dotenv`
- `streamlit-ext==0.1.19`

The most important one is Streamlit itself, since it provides the entire UI layer.

### 6.14 Maintenance considerations

Most frontend work in the future will probably happen in `frontend/main.py`. Typical changes might include:

- adjusting layout and visuals,
- improving status messages and error reporting,
- adding input validation or clearer user guidance,
- supporting additional result visualisations,
- previewing uploaded images or processed outputs,
- better handling of large file batches.

A few implementation details are worth understanding before changing anything:

- The file uploader reset works by changing the Streamlit widget key through session state.
- The displayed results depend on the backend returning `density_info` fields in an order consistent with the DataFrame column list.
- Backend communication relies entirely on `API_URL` being correct.
- The progress display assumes files are processed sequentially, one request per image.

For anyone maintaining the frontend, the good news is that the logic is compact and mostly linear. The key thing to remember is that it is a presentation layer only. Any changes to scientific calculations, segmentation behaviour, mask interpretation, or returned metadata should go in the backend.

## 7 Backend

### 7.1 Role of the backend

The backend is the computational core of StomaML. It takes uploaded images from the frontend, loads the trained segmentation model, runs inference, computes derived measurements, and returns everything as structured JSON. In short, the backend is where all the scientifically relevant computation happens.

Its job can be broken down into four steps. First, it loads the trained YOLO segmentation model at startup. Second, it exposes API endpoints through FastAPI so the frontend can send requests. Third, it runs inference on uploaded microscope images and post-processes the segmentation results into structured measurements. Fourth, it returns only the fields that the frontend and downloadable outputs actually need. This keeps the frontend lightweight and the analysis logic in one place.

### 7.2 Backend files

The backend code lives in `backend/`. The main files are:

- `main.py` – defines the FastAPI application, sets up middleware, loads the model at startup, and exposes the API endpoints.
- `utils/inference.py` – the inference and post-processing pipeline. This file turns raw YOLO outputs into the JSON-compatible result structure that the API returns.
- `utils/model_utils.py` – the `ModelManager` class, responsible for loading the model once and reusing it across requests.
- `models/weights.pt` – the trained model weights needed at runtime.
- `requirements.txt` – backend Python dependencies.

- `Dockerfile` – the backend container image definition for local and deployed use.

The division of work is simple: `main.py` handles the application lifecycle and HTTP layer, `model_utils.py` handles model access, and `inference.py` handles image processing and measurement logic.

## 7.3 FastAPI application structure

The backend is defined in `backend/main.py`. It uses FastAPI with a lifespan function to make sure the model gets loaded during startup rather than on the first prediction request.

Logging is set to `INFO` level. A global `ModelManager` instance is created at import time, and the lifespan function then calls:

```
model_manager.load_model("models/weights.pt")
```

If this fails, the backend does not start at all. This is intentional – it makes problems like missing files or incompatible weights show up immediately rather than only when someone tries to run their first analysis.

The FastAPI app is created with the title `Stoma Segmentation API`. It also uses trusted host middleware to restrict which host headers it accepts.

## 7.4 Trusted host middleware

The backend adds `TrustedHostMiddleware` with a specific list of allowed hosts. Currently that list includes:

- `stoma-ml-api-stoma-ml.apps.ocp-test-0.k8s.it.helsinki.fi`
- `stoma-ml-api-stoma-ml.apps.ocp-bm-0.k8s.it.helsinki.fi`
- `localhost`
- `127.0.0.1`
- `backend`

This is especially relevant for deployment. If the backend ever gets exposed under a new hostname and that hostname is not added here, requests will be rejected even if the service itself is running fine. So this is one of the first things to update after any hostname change.

## 7.5 API endpoints

The backend has three endpoints.

`GET /` Returns a simple status message:

```
{"status": "healthy", "message": "Stoma Segmentation API is running"}
```

It is mainly there as a quick check that the service is up.

`GET /health` Returns:

```
{"status": "healthy", "model_loaded": ...}
```

where `model_loaded` is a boolean from `model_manager.is_loaded()`. More informative than the root endpoint, because it tells you not just that the process is running, but also whether the model loaded successfully.

`POST /predict` The main inference endpoint. It accepts:

- one or more uploaded files via `files: List[UploadFile]`,
- a confidence threshold via the query parameter `conf`,
- an image scale via the query parameter `scale`.

Parameter validation in FastAPI:

- `conf`: default 0.25, range $0.0 \leq$ `conf` $\leq 1.0$,
- `scale`: default 10/46, must be strictly greater than 0.

Before running inference, the endpoint checks a few things:

- if the model is not loaded, it returns HTTP 503,
- if no files were provided, it returns HTTP 400,
- if all uploaded files are empty, it returns HTTP 400.

It then reads the file contents into memory, stores them as `(filename, bytes)` tuples, calls the batch inference pipeline, and returns a cleaned response containing only the fields meant for downstream use.

## 7.6 Returned API response structure

The `/predict` endpoint returns JSON with this top-level structure:

```
{"total_images": ..., "results": [...]}
```

Each entry in `results` contains:

- `filename`
- `index`
- `success`
- `density_info`
- `stomata`

This explicit cleaning step in `main.py` prevents accidental leakage of internal fields if the inference code changes later. The frontend depends only on this cleaned format, not on every internal detail of the processing pipeline.

## 7.7 Model manager

Model loading is wrapped in the `ModelManager` class in `utils/model_utils.py`. It provides a small abstraction over the underlying `YOLO` model object.

It contains:

- a private lock (`_lock`),
- a private model reference (`_model`),
- `load_model(weights_path)`,
- `is_loaded()`,
- `get_model()`.

The lock makes model initialisation thread-safe. The main purpose of the class is to guarantee the model is loaded once and then reused for all subsequent requests. This avoids repeated model construction and keeps route handlers simple.

For future maintainers: route functions should not create YOLO models directly. Always go through the manager.

## 7.8 Inference pipeline

The core inference logic is in `utils/inference.py`. At a high level, the pipeline does the following:

1. raw uploaded image bytes are decoded into an RGB image using Pillow,
2. the image is passed to the YOLO segmentation model,
3. the prediction result is post-processed into binary masks and metadata,
4. stomata-level records and image-level density statistics are computed,
5. the result is returned as a JSON-compatible Python dictionary.

The YOLO model is called with:

```
model(image, conf=conf_threshold, imgsz=640, retina_masks=True, rect=False, iou=0.25)[0]
```

So inference currently uses image size 640, retina-quality masks, no rectangular inference, and an IoU threshold of 0.25. If model or deployment requirements change down the line, these are among the most likely parameters to revisit.

## 7.9 Mask handling and post-processing

The raw YOLO segmentation output is not returned as-is. Instead, `process_detections()` transforms it into a more stable, application-specific structure.

For each predicted mask, the pipeline:

- retrieves the floating-point mask array,
- retrieves the associated confidence and predicted class,
- thresholds the mask at 0.5 to get a binary mask,
- resizes it to the original image dimensions if needed (nearest-neighbour interpolation),
- checks whether the class is non-countable,
- if non-countable, merges the mask into the global non-countable mask,
- if the class is `stomata`, stores it as a candidate for further filtering.

The non-countable classes are currently:

```
NON_COUNTABLE_CLASSES = {"nothing", "trichome", "vein"}
```

After the global non-countable mask is built, the countable region is defined as every pixel not covered by it. Each stomata candidate is then checked: it is only included in the final results if at least

```
COUNTABLE_OVERLAP_THRESHOLD = 0.5
```

of its mask area lies within countable area. Candidates that overlap too much with non-countable regions get excluded from the stomata list and therefore also from density calculations.

This is an important point for maintenance. If a future model uses different class names, or if the definition of countable area changes, both `NON_COUNTABLE_CLASSES` and the overlap threshold may need updating. Otherwise the reported counts and densities might not match the intended biological interpretation.

## 7.10 Image-level calculations

After all masks are processed, the backend computes an image-level summary called `density_info`, containing the fields described in Section 3. These are derived from the original image dimensions, the union of non-countable masks, the number of accepted stomata, and the user-provided scale. Density in $mm^2$ is computed by the helper `density_per_mm2()`, which converts the countable pixel area to square millimetres and then divides stomata count by that area.

## 7.11 Stomata-level calculations

For each accepted stomata, the backend creates a metadata dictionary using `build_metadata()`, containing the per-stomata fields described in Section 3. The implementation details worth highlighting here are:

- Stomata length is estimated by fitting an ellipse to the binary mask via OpenCV's `cv2.fitEllipse`. The major axis is used as the length estimate. If the mask has fewer than five points, ellipse fitting cannot be done and the length values are `None`. This is an approximation, not a direct biological measurement.
- Nearest-neighbour distances are computed by `add_closest_stomata_distance()` after all metadata entries have been created. These are centroid-to-centroid Euclidean distances, not edge-to-edge. If only one stomata is present, the distance fields are `None`.
- The `fraction_in_countable_area` field serves a dual purpose: it is both a diagnostic value and a record of why a particular stomata was included.

## 7.12 Functions in `utils/inference.py`

The main helper functions in the inference module are:

- `mask_to_bbox()` – computes a tight bounding box around a binary mask.
- `get_ellipse_length()` – fits an ellipse to mask pixels and returns the major and minor axis lengths.
- `build_metadata()` – creates one per-stomata metadata dictionary.
- `add_closest_stomata_distance()` – computes nearest-neighbour distances across all stomata in an image.
- `density_per_mm2()` – converts countable area to $mm^2$ and computes density.
- `process_detections()` – transforms one raw YOLO result into stomata metadata and density summary, builds the non-countable mask, and filters stomata candidates by their overlap with countable area.
- `run_inference()` – end-to-end inference for a single image.
- `batch_inference()` – loops over a list of uploaded images and calls `run_inference()` on each.

Despite the name `batch_inference`, the implementation is sequential: images are processed one after another in a loop. This matches the frontend, which also sends files one at a time.

## 7.13 Error handling

The API-level error codes (HTTP 503 for missing model, HTTP 400 for missing or empty files) were already described in the API endpoints subsection. Beyond that, `run_inference()` wraps the processing in a `try/except` block, logs the exception, and re-raises it. So inference errors show up in backend logs rather than being silently ignored.

In practice, debugging usually involves checking container or server logs when the frontend only reports that a request failed. That is expected – the backend logs are where the useful details are.

## 7.14 Runtime defaults and constants

The two key constants `NON_COUNTABLE_CLASSES` and `COUNTABLE_OVERLAP_THRESHOLD` were already introduced in the mask handling subsection above. Having them as named constants makes it fairly easy to adapt the backend if a future model uses different class labels or if the overlap rule needs changing.

## 7.15 Dependencies

Backend dependencies are in `backend/requirements.txt`. The main ones are:

- `fastapi==0.128.2`
- `uvicorn==0.40.0`
- `python-multipart`
- `pillow`
- `numpy`
- `ultralytics==8.4.12`
- `opencv-python-headless`

The Docker image also explicitly installs CPU-only PyTorch and Torchvision before the rest of the dependencies. This matters because `ultralytics` depends on PyTorch, and the container is set up for CPU inference rather than GPU.

Any major upgrade to `ultralytics`, `torch`, or `opencv` should be tested carefully since it could affect model loading, output format, or mask handling.

## 7.16 Containerisation

The backend has its own `Dockerfile`, built from `python:3.10-slim`. The build process:

1. sets `/app` as the working directory,
2. installs system libraries needed by the model stack and OpenCV,
3. copies `requirements.txt`,
4. installs CPU-only PyTorch and Torchvision from the PyTorch CPU wheel index,
5. installs the Python dependencies from `requirements.txt`,
6. copies `main.py`,
7. copies the `utils/` directory,
8. creates a `models/` directory and copies `models/weights.pt`,
9. exposes port 8080,
10. starts Uvicorn on port 8080.

### 7.17 Maintenance considerations

Most backend work in the future will likely be in `backend/main.py` and especially `backend/utils/inference.py`. Common tasks might include:

- updating the model weights,
- adapting to changed class names or mappings,
- changing post-processing logic,
- adding new output fields,
- improving validation and error reporting,
- optimising inference performance,
- introducing automated tests for output consistency.

A few key assumptions to keep in mind (most already discussed in the subsections above):

- The backend assumes YOLO mask outputs can be thresholded at 0.5 and treated as binary segmentation masks.
- The distinction between stomata and non-countable area depends entirely on the class names the model predicts.
- The frontend expects the response to contain `density_info` and `stomata` fields with specific names.

Since the backend is the scientific core, even small logic changes here can alter the reported biological measurements. Any such change should be documented and tested end-to-end with the frontend.

## 8 Deployment

### 8.1 OpenShift

The deployed version runs on OpenShift in containers. The frontend and backend are separate services, each with its own container image and runtime configuration. Communication between them depends on correct routing and environment settings.

This section documents the application-specific deployment assumptions for running StomaML on OpenShift. It is not a guide to OpenShift platform administration in general. For more information, see the official OpenShift documentation: https://docs.redhat.com/en/documentation/openshift_container_platform/4.21.

From the application's perspective, the key deployment facts are as follows:

- the frontend and backend are deployed separately,
- the frontend needs the correct backend prediction endpoint,
- the backend needs access to the model weights file at runtime,
- hostname-related backend settings must match the deployed route.

One important detail: the frontend gets the backend URL as a secret. Because the frontend code expects a full prediction endpoint, this value should point to the `/predict` route, not just the backend root URL.

On the backend side, the model weights must be available at runtime. The expected path inside the backend project is:

`models/weights.pt`

Without this file, the backend will fail at startup since the model is loaded when FastAPI starts.

Also remember that the backend uses trusted host middleware. If the backend route hostname changes, the allowed host list has to be updated too. Otherwise requests get rejected even though the service is running.

A practical way to think about deployment is that three things all need to be right at the same time:

1. the containers start successfully,
2. the frontend points to the correct backend endpoint,
3. the backend accepts requests for the deployed hostname and has access to the model weights.

If any one of these is off, the deployment will look broken even if the other parts work fine.

### 8.2 Local use and testing

Although OpenShift is the main deployment target, local execution is still important for development, testing, and debugging. The essentials are the same: backend dependencies installed, model weights present, backend starts, and the frontend points to the right endpoint.

For most people, Docker Compose is the easiest option because it avoids the manual setup of running two services separately. The local Compose setup is also useful because it mirrors the deployed architecture reasonably well – frontend and backend still run as separate services talking over HTTP.

## 9 Checks after changes

After any meaningful change, it is a good idea to run a short end-to-end test. This goes for both code changes and deployment changes, since configuration issues often only show up when you exercise the full system.

A minimal check should include:

1. confirm that the backend starts successfully,
2. verify that the backend health endpoint responds,
3. launch the frontend,
4. verify that the frontend can reach the backend,
5. run a test analysis on one or more sample images,
6. confirm that the results table and downloadable files look reasonable.

For local development, that typically means checking the health endpoint first and then opening the frontend in a browser. For OpenShift, it means verifying both the frontend and backend routes, then doing at least one real analysis through the frontend.

The most likely failure points after deployment changes are:

- wrong backend URL in the frontend configuration,
- missing or inaccessible model weights,
- hostname changes not reflected in trusted host settings,
- services starting successfully but without actual end-to-end connectivity.

A deployment should not be considered done just because the containers are running. At least one full analysis should be tested after meaningful changes.